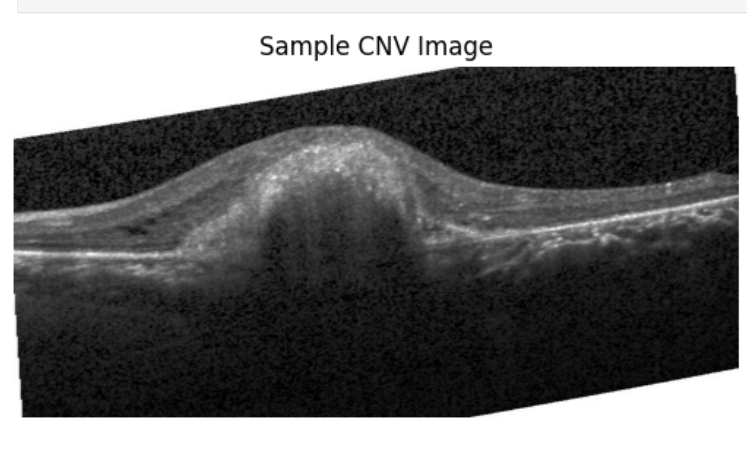
Loading the unhealthy dataset

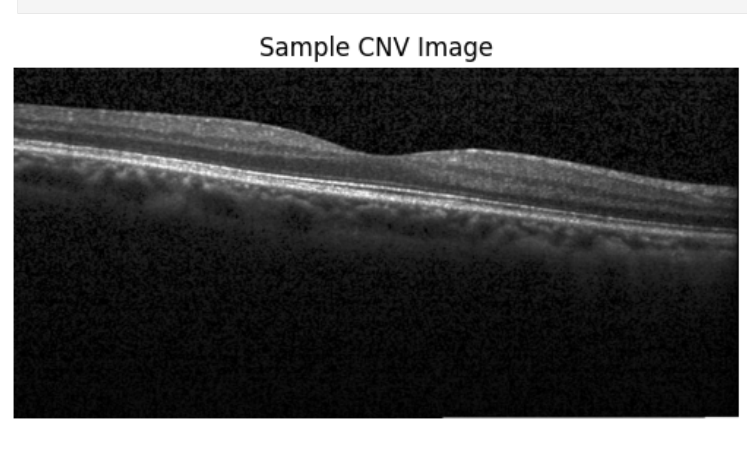
In [1]:
```python
# Import necessary libraries
import cv2
import os
from PIL import Image
import time
import matplotlib.pyplot as plt
import numpy as np
```

In [2]:
```python
import cv2, os
from pathlib import Path

folder_path = Path("Data/CellData/OCT/test/CNV")
images_unhealthy = []

for file in folder_path.iterdir():
    if file.suffix.lower() in {".png", ".jpg", ".jpeg"}:
        img = cv2.imread(str(file), cv2.IMREAD_GRAYSCALE)  # returns a NumPy array
        if img is not None:
            images_unhealthy.append(img)

plt.imshow(images_unhealthy[3], cmap='gray')
plt.axis('off')
plt.title('Sample CNV Image')
plt.show()
```



Loading the healthy dataset

In [3]:
```python
import cv2, os
from pathlib import Path

folder_path = Path("Data/CellData/OCT/test/Normal")
images_healthy = []

for file in folder_path.iterdir():
    if file.suffix.lower() in {".png", ".jpg", ".jpeg"}:
        img = cv2.imread(str(file), cv2.IMREAD_GRAYSCALE)  # returns a NumPy array
        if img is not None:
            images_healthy.append(img)

plt.imshow(images_healthy[3], cmap='gray')
plt.axis('off')
plt.title('Sample CNV Image')
plt.show()
```



Loading train Healthy

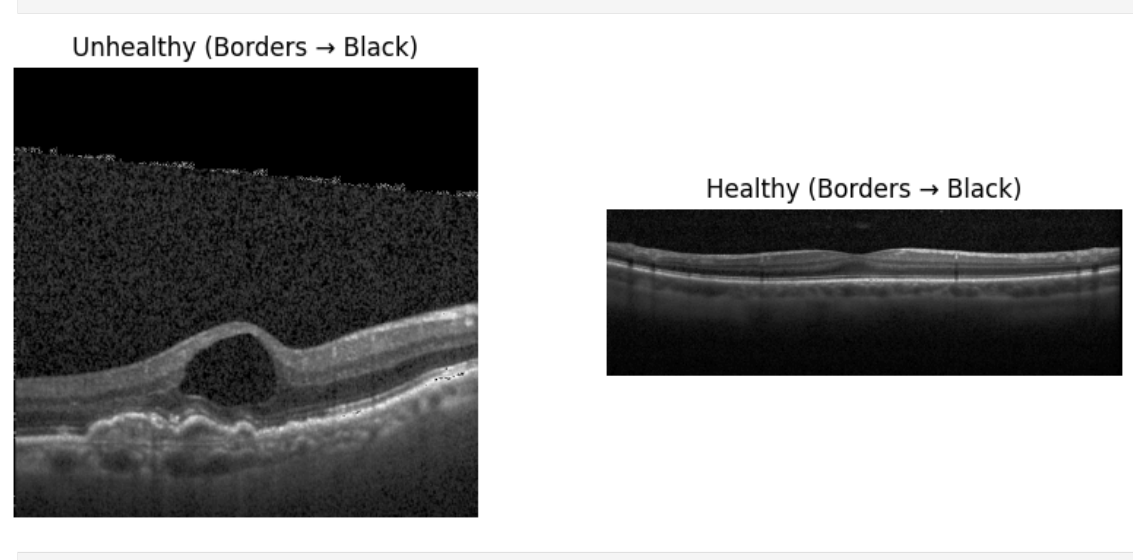
In [4]:
```python
import cv2, os
from pathlib import Path

folder_path = Path("Data/CellData/OCT/train/Normal")
images_healthy_train = []

for file in folder_path.iterdir():
    if file.suffix.lower() in {".png", ".jpg", ".jpeg"}:
        img = cv2.imread(str(file), cv2.IMREAD_GRAYSCALE)  # returns a NumPy array
        if img is not None:
            images_healthy_train.append(img)
```

train unhealthy

In [5]:
```python
import cv2, os
from pathlib import Path

folder_path = Path("Data/CellData/OCT/train/CNV")
images_unhealthy_train = []

for file in folder_path.iterdir():
    if file.suffix.lower() in {".png", ".jpg", ".jpeg"}:
        img = cv2.imread(str(file), cv2.IMREAD_GRAYSCALE)  # returns a NumPy array
        if img is not None:
            images_unhealthy_train.append(img)
```

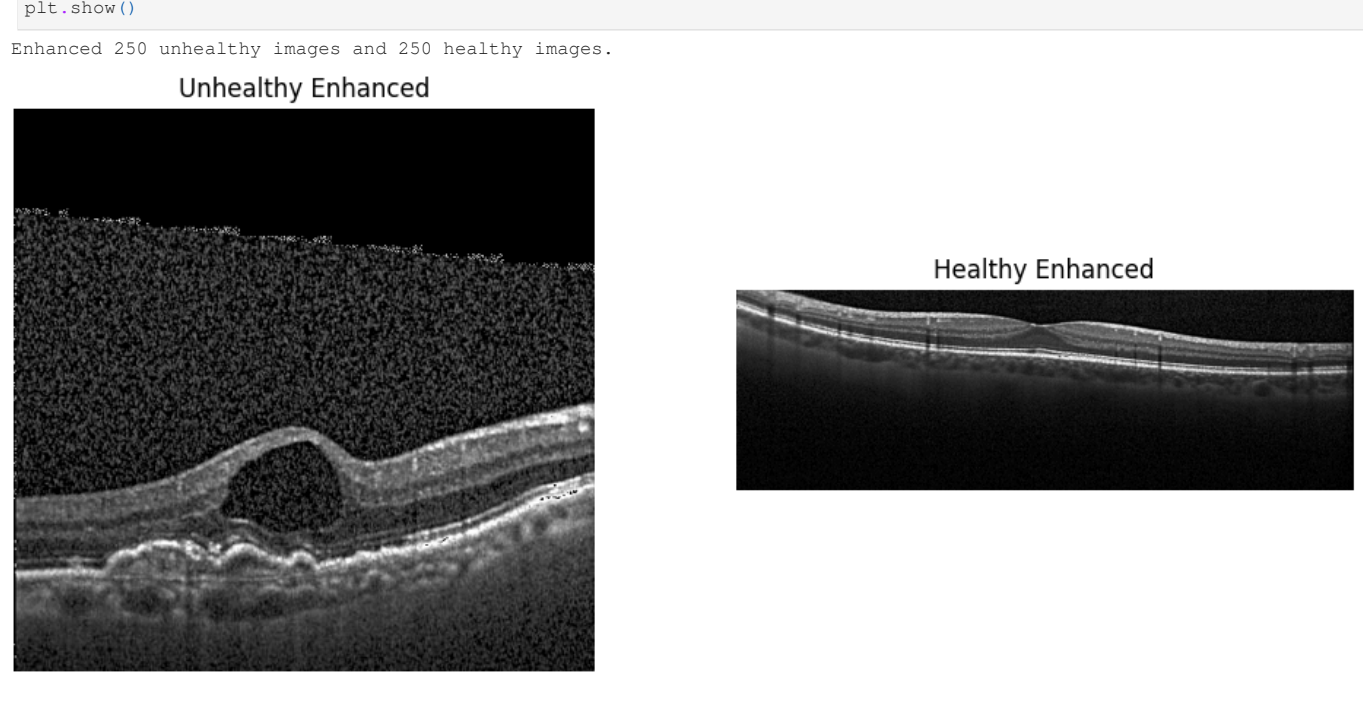
Remove borders

In [ ]:
```python
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

# Function to replace white border pixels with black
def remove_white_borders(arr, threshold=250):
    """
    Replace white pixels (>= threshold) with black (0).
    Works for 2D grayscale images.
    """
    arr = arr.copy()
    mask = arr >= threshold
    arr[mask] = 0
    return arr

# Modify the existing arrays directly
for i in range(len(images_unhealthy)):
    images_unhealthy[i] = remove_white_borders(images_unhealthy[i])

for i in range(len(images_healthy)):
    images_healthy[i] = remove_white_borders(images_healthy[i])

for i in range(len(images_unhealthy_train)):
    images_unhealthy_train[i] = remove_white_borders(images_unhealthy_train[i])

for i in range(len(images_healthy_train)):
    images_healthy_train[i] = remove_white_borders(images_healthy_train[i])


# Show before and after for one example
idx = 4
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(images_unhealthy_train[idx], cmap='gray')
plt.axis('off')
plt.title('Unhealthy (Borders → Black)')

plt.subplot(1, 2, 2)
plt.imshow(images_healthy_train[idx], cmap='gray')
plt.axis('off')
plt.title('Healthy (Borders → Black)')

plt.show()
```



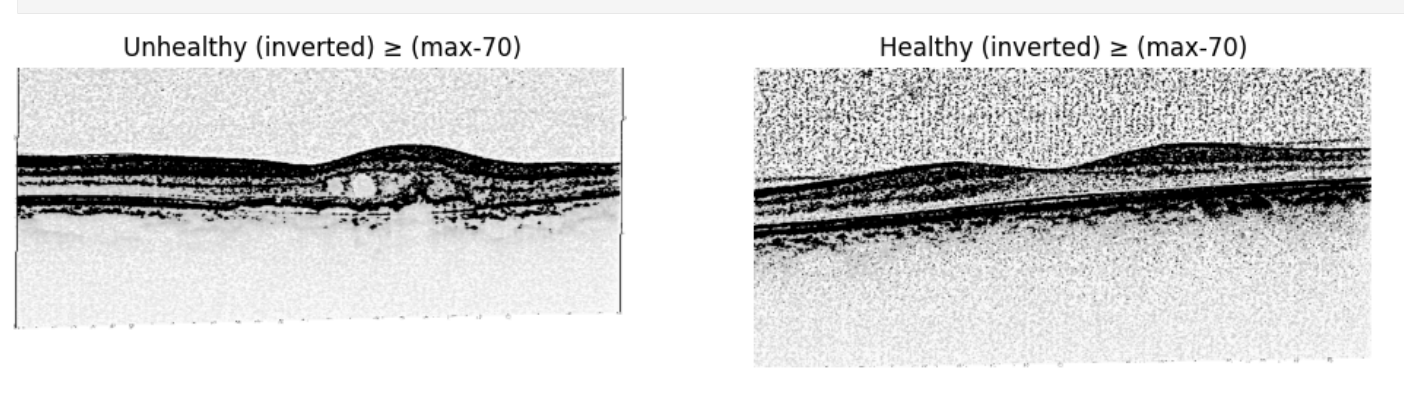
In [ ]:
```python
import cv2
import numpy as np,gc
from PIL import Image
import matplotlib.pyplot as plt

def _ensure_odd_kernel(k):
    # Make sure Gaussian kernel size is valid (odd, >=1)
    if isinstance(k, int):
        return (k if k % 2 == 1 else k + 1)
    kh = k[0] if isinstance(k, tuple) else k
    kw = k[1] if isinstance(k, tuple) else k
    kh = kh if kh % 2 == 1 else kh + 1
    kw = kw if kw % 2 == 1 else kw + 1
    return (max(1, kh), max(1, kw))

def _to_gray_array(img):
    """Accepts np.ndarray (gray or RGB/BGR) or PIL.Image; returns 2D uint8 array."""
    if isinstance(img, np.ndarray):
        if img.ndim == 2:
            return img.astype(np.uint8, copy=False)
        elif img.ndim == 3:
            # Assume BGR (cv2) or RGB; if unsure, convert by averaging channels
            # Safer universal path: convert to gray with cv2 (assumes BGR)
            try:
                return cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            except cv2.error:
                # Fallback: average channels
                return np.clip(img.mean(axis=2), 0, 255).astype(np.uint8)
    else:
        # PIL.Image
        return np.array(img.convert("L"), dtype=np.uint8)

def enhance_low_high_np(img, blur_kernel=(27, 27), highpass_weight=1.0, keep_type='ndarray'):
    """
    Apply Gaussian low-pass + high-pass enhancement.
    Accepts: np.ndarray or PIL.Image
    Returns: np.ndarray (grayscale) by default; set keep_type='pil' to get PIL.Image
    """
    k = _ensure_odd_kernel(blur_kernel)
    arr = _to_gray_array(img).astype(np.float32)

    # Low-pass (Gaussian blur)
    blurred = cv2.GaussianBlur(arr, k, 0)

    # High-pass
    highpass = arr - blurred

    # Combine
    sharpened = cv2.addWeighted(arr, 1.0, highpass, float(highpass_weight), 0)

    # Clip back to valid range
    sharpened = np.clip(sharpened, 0, 255).astype(np.uint8)

    if keep_type == 'pil':
        return Image.fromarray(sharpened)
    return sharpened

# Apply to all images (lists may contain NumPy arrays now)
images_unhealthy_enhanced       = [enhance_low_high_np(img) for img in images_unhealthy]
del images_unhealthy
gc.collect()
images_healthy_enhanced         = [enhance_low_high_np(img) for img in images_healthy]
del images_healthy
gc.collect()
images_unhealthy_train_enhanced = [enhance_low_high_np(img) for img in images_unhealthy_train]
del images_unhealthy_train
gc.collect()
images_healthy_train_enhanced   = [enhance_low_high_np(img) for img in images_healthy_train]
del images_healthy_train
gc.collect()

print(f"Enhanced {len(images_unhealthy_enhanced)} unhealthy images and {len(images_healthy_enhanced)} healthy images.")

# Show one example from each
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(images_unhealthy_train_enhanced[5], cmap='gray')
plt.axis('off')
plt.title('Unhealthy Enhanced')

plt.subplot(1, 2, 2)
plt.imshow(images_healthy_train_enhanced[5], cmap='gray')
plt.axis('off')
plt.title('Healthy Enhanced')

plt.show()
```

```
Enhanced 250 unhealthy images and 250 healthy images.
```



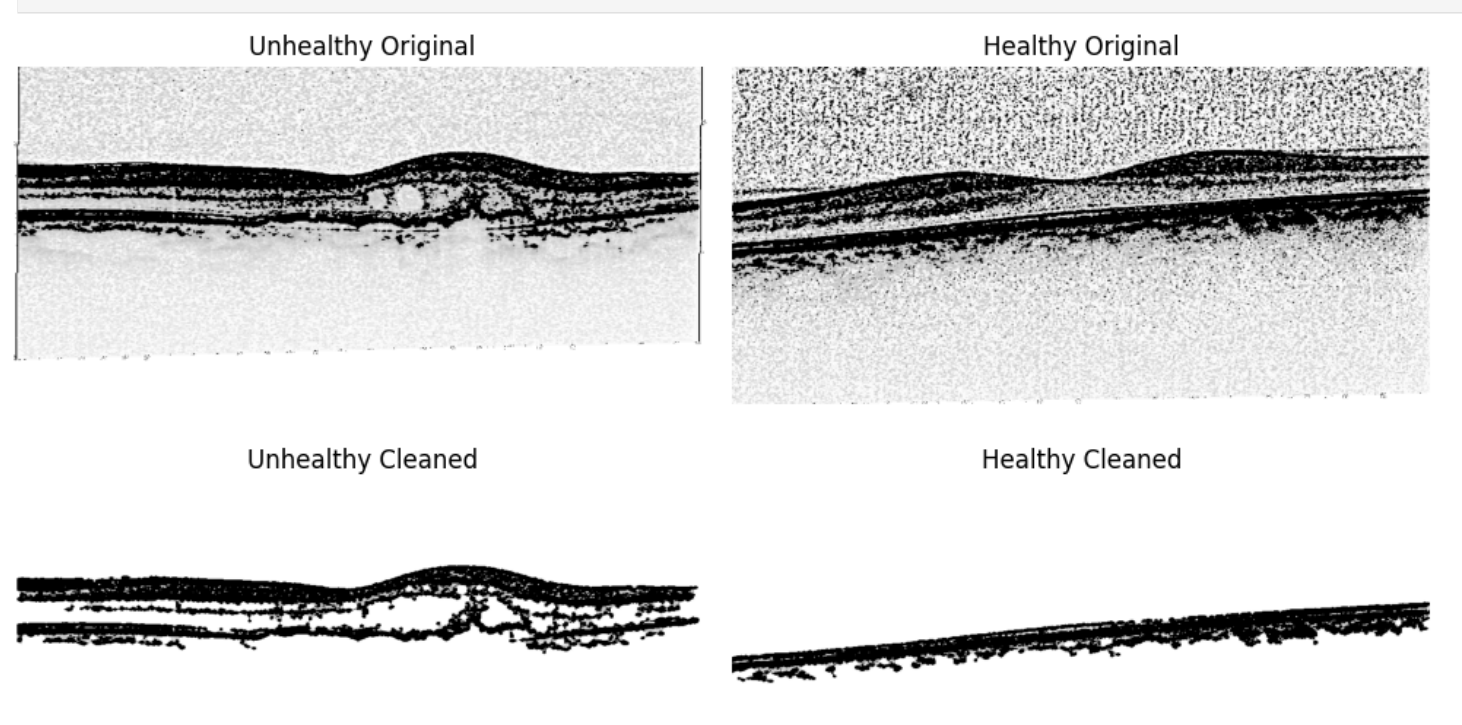
Reverse colors to detect black pockets

In [8]:
```python
import numpy as np, gc
import matplotlib.pyplot as plt
import PIL.Image

def keep_inverted_top_band(pil_list, delta=20):
    """
    For each PIL 'L' image:
      - convert to grayscale ndarray
      - invert: new_pixel = 255 - old_pixel
      - find max intensity in inverted image
      - threshold at max - delta
      - keep original inverted values where >= threshold, else 0 (black)
    Returns list of uint8 numpy arrays.
    """
    out = []
    for im in pil_list:
        # ensure grayscale ndarray
        if isinstance(im, PIL.Image.Image):
            g = np.array(im.convert("L"), dtype=np.uint8)
        else:
            g = np.array(im, dtype=np.uint8)

        # invert the grayscale image
        inv = 255 - g

        # compute threshold
        maxv = int(inv.max())
        thr = max(0, maxv - int(delta))

        # keep only bright (white) pixels in inverted image
        keep = (inv >= thr)
        result = np.where(keep, inv, 0).astype(np.uint8)
        out.append(result)
    return out

# Apply to your variables
unhealthy_inv_top = keep_inverted_top_band(images_unhealthy_enhanced, delta=70)
del images_unhealthy_enhanced
gc.collect()
healthy_inv_top   = keep_inverted_top_band(images_healthy_enhanced,   delta=70)
del images_healthy_enhanced
gc.collect
unhealthy_inv_top_train = keep_inverted_top_band(images_unhealthy_train_enhanced, delta=70)
del images_unhealthy_train_enhanced
gc.collect()
healthy_inv_top_train   = keep_inverted_top_band(images_healthy_train_enhanced,   delta=70)
del images_healthy_train_enhanced
gc.collect()

# Quick visual check
idx = 6
plt.figure(figsize=(12,5))
plt.subplot(1,2,1); plt.imshow(unhealthy_inv_top[idx], cmap='gray'); plt.axis('off'); plt.title('Unhealthy (inverted) ≥ (max-70)')
plt.subplot(1,2,2); plt.imshow(healthy_inv_top[idx],   cmap='gray'); plt.axis('off');  plt.title('Healthy (inverted) ≥ (max-70)')
plt.show()
```


Unhealthy (inverted) ≥ (max-70)
Healthy (inverted) ≥ (max-70)

noise removal

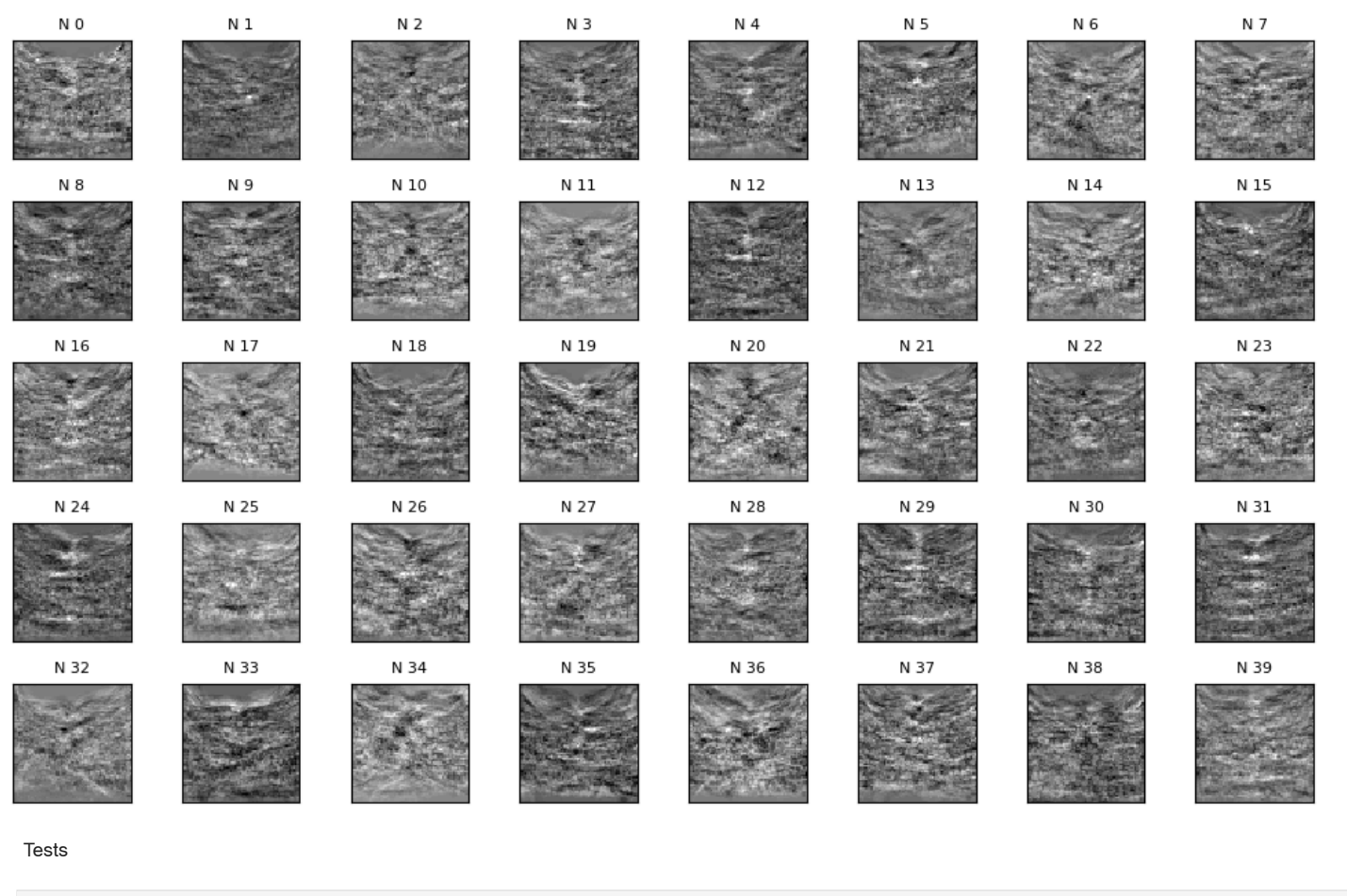
In [ ]:
```python
import cv2
import numpy as np, gc
import matplotlib.pyplot as plt
from PIL import Image

def keep_main_object_preserve_intensity_batch(pil_list, thresh_val=110, morph_kernel=5, return_array=True):
    """
    Apply 'keep_main_object_preserve_intensity' to every image in pil_list.
    Accepts list/array of images (NumPy or PIL), returns list or stacked array.
    """
    out = []

    # structuring element once
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (morph_kernel, morph_kernel))

    for im in pil_list:
        # --- normalize input to grayscale uint8 NumPy array ---
        if isinstance(im, Image.Image):         # PIL
            if im.mode != "L":
                im = im.convert("L")
            img = np.array(im)
        else:
            img = np.asarray(im)
            if img.ndim == 3:                   # RGB/BGR -> gray
                # assume RGB ordering; if your data is BGR, swap code to COLOR_BGR2GRAY
                img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        img = img.astype(np.uint8)

        # threshold dark regions
        _, mask = cv2.threshold(img, thresh_val, 255, cv2.THRESH_BINARY_INV)

        # morphological cleanup
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k, iterations=1)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k, iterations=2)

        # keep largest component only
        num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(mask, connectivity=8)
        if num_labels > 1:
            largest_label = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
            mask = np.where(labels == largest_label, 255, 0).astype(np.uint8)
        else:
            # only background or one component; ensure mask is binary {0,255}
            mask = (labels == 1).astype(np.uint8) * 255 if num_labels == 2 else mask

        # preserve interior, set outside to 255
        result = np.where(mask == 255, img, 255).astype(np.uint8)
        out.append(result)

    if return_array and isinstance(pil_list, np.ndarray):
        return np.stack(out)
    return out

idx = 6
fig, axes = plt.subplots(2, 2, figsize=(10, 6))
axes[0,0].imshow(unhealthy_inv_top[idx], cmap='gray');          axes[0,0].set_title("Unhealthy Original"); axes[0,0].axis('off')
axes[0,1].imshow(healthy_inv_top[idx], cmap='gray');            axes[0,1].set_title("Healthy Original");   axes[0,1].axis('off')

# Process all splits
unhealthy_processed_all       = keep_main_object_preserve_intensity_batch(unhealthy_inv_top)
del unhealthy_inv_top
gc.collect()
healthy_processed_all         = keep_main_object_preserve_intensity_batch(healthy_inv_top)
del healthy_inv_top
gc.collect()
unhealthy_train_processed_all = keep_main_object_preserve_intensity_batch(unhealthy_inv_top_train)
del unhealthy_inv_top_train
gc.collect()
healthy_train_processed_all   = keep_main_object_preserve_intensity_batch(healthy_inv_top_train)
del healthy_inv_top_train
gc.collect()

# Plot a single index for before/after

axes[1,0].imshow(unhealthy_processed_all[idx], cmap='gray');    axes[1,0].set_title("Unhealthy Cleaned");  axes[1,0].axis('off')
axes[1,1].imshow(healthy_processed_all[idx], cmap='gray');      axes[1,1].set_title("Healthy Cleaned");    axes[1,1].axis('off')
plt.tight_layout()
plt.show()
```



Modell

In [10]:
```python
import warnings
import numpy as np
import matplotlib.pyplot as plt
from sklearn.exceptions import ConvergenceWarning
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

import cv2
from collections import Counter

# =========================
# Config
# =========================
TARGET_SIZE = (64, 64)      # try (96,96) if you need more detail
EPOCHS = 30
BATCH_SIZE = 512
HIDDEN = (256, 128)
LEARNING_RATE = 1e-3
RANDOM_SEED = 1

# =========================
# Helpers
# =========================
def most_common_hw(imgs):
    sizes = [np.asarray(im).shape[:2] for im in imgs]
    (H, W), _ = Counter(sizes).most_common(1)[0]
    return int(H), int(W)

def to_array_uniform(imgs, mode="resize", target_size=None, pad_value=255):
    """
    Convert images to a stacked uint8 array with uniform (H,W).
    mode:
      - "resize": warp to target_size (no cropping). Uses INTER_AREA for downscale.
      - "pad":    paste into top-left of a target canvas (no warping).
    Converts to grayscale if needed.
    """
    if isinstance(imgs, np.ndarray) and imgs.ndim >= 3:
        seq = [imgs[i] for i in range(len(imgs))]
    else:
        seq = list(imgs)

    if target_size is None:
        if mode == "resize":
            target_size = most_common_hw(seq)
        else:
            Hs, Ws = zip(*[np.asarray(im).shape[:2] for im in seq])
            target_size = (max(Hs), max(Ws))
    Ht, Wt = target_size

    out = []
    for im in seq:
        im = np.asarray(im)
        # force grayscale (H,W)
        if im.ndim == 3:
            if im.shape[2] == 3:
                im = cv2.cvtColor(im, cv2.COLOR_RGB2GRAY)
            elif im.shape[2] == 4:
                im = cv2.cvtColor(im, cv2.COLOR_RGBA2GRAY)
            else:
                im = im.mean(axis=2).astype(im.dtype)

        if mode == "resize":
            interp = cv2.INTER_AREA if (im.shape[0] > Ht or im.shape[1] > Wt) else cv2.INTER_LINEAR
            im_u = cv2.resize(im, (Wt, Ht), interpolation=interp)
        elif mode == "pad":
            im_u = np.full((Ht, Wt), pad_value, dtype=im.dtype)
            h, w = im.shape[:2]
            im_u[:h, :w] = im
        else:
            raise ValueError("mode must be 'resize' or 'pad'")
        out.append(im_u)

    return np.stack(out)  # uint8

def batch_iter(X, y=None, batch_size=512, shuffle=False, rng=None, mean_img=None):
    """
    Yield batches as float32 in [0,1] (optionally mean-centered), flattened.
    X stays uint8 on disk/memory until here.
    """
    n = len(X)
    idx = np.arange(n)
    if shuffle:
        rng = rng or np.random.default_rng()
        rng.shuffle(idx)
    for start in range(0, n, batch_size):
        sel = idx[start:start+batch_size]
        xb = X[sel].astype(np.float32) / 255.0
        if mean_img is not None:
            xb = xb - mean_img  # (1,H,W) broadcast
        xb = xb.reshape(len(xb), -1)
        if y is None:
            yield xb
        else:
            yield xb, y[sel]

def coef_to_image(coef_vector, H, W, C):
    if C == 1:
        return coef_vector.reshape(H, W)
    return coef_vector.reshape(H, W, C).mean(axis=2)

# =========================
# Load + unify sizes
# (expects you already have:
#  unhealthy_train_processed_all, healthy_train_processed_all,
#  unhealthy_processed_all, healthy_processed_all)
# =========================
unhealthy_train_arr = to_array_uniform(unhealthy_train_processed_all, mode="resize", target_size=TARGET_SIZE)
healthy_train_arr   = to_array_uniform(healthy_train_processed_all,   mode="resize", target_size=TARGET_SIZE)
unhealthy_test_arr  = to_array_uniform(unhealthy_processed_all,       mode="resize", target_size=TARGET_SIZE)
healthy_test_arr    = to_array_uniform(healthy_processed_all,         mode="resize", target_size=TARGET_SIZE)

uh_train = unhealthy_train_arr
h_train  = healthy_train_arr
uh_test  = unhealthy_test_arr
h_test   = healthy_test_arr

# Sanity checks
assert uh_train.ndim == 3 and h_train.ndim == 3, "Expect grayscale (N,H,W)"
assert uh_test.ndim  == 3 and h_test.ndim  == 3, "Expect grayscale (N,H,W)"
assert uh_train.shape[1:3] == h_train.shape[1:3], "Train sizes must match"
assert uh_test.shape[1:3]  == h_test.shape[1:3],  "Test sizes must match"

H, W = uh_train.shape[1:]
C = 1
D = H * W * C

# =========================
# Build X/y (keep images separate; no giant float arrays)
# =========================
X_train_img = np.concatenate([h_train, uh_train], axis=0)  # uint8
y_train = np.concatenate([
    np.zeros(len(h_train), dtype=np.int64),
    np.ones(len(uh_train), dtype=np.int64)
])

X_test_img = np.concatenate([h_test, uh_test], axis=0)
y_test = np.concatenate([
    np.zeros(len(h_test), dtype=np.int64),
    np.ones(len(uh_test), dtype=np.int64)
])

print(f"Train: {len(X_train_img)} | Test: {len(X_test_img)} | Input dim: {D}")

# Per-pixel mean on a subset (keeps memory small)
subset = min(len(X_train_img), 5000)
mean_img = (X_train_img[:subset].astype(np.float32) / 255.0).mean(axis=0, keepdims=True)  # (1,H,W)

# =========================
# Model
# =========================
mlp = MLPClassifier(
    hidden_layer_sizes=HIDDEN,
    solver="adam",
    alpha=1e-4,
    learning_rate_init=LEARNING_RATE,
    random_state=RANDOM_SEED,
    max_iter=1,      # we'll loop epochs manually
    warm_start=True, # keep weights between .partial_fit calls
    verbose=False,
)

classes = np.array([0, 1], dtype=np.int64)
rng = np.random.default_rng(RANDOM_SEED)

# ---- Seed the model ONCE with a batch containing both classes
i0 = np.where(y_train == 0)[0]
i1 = np.where(y_train == 1)[0]
n_half = max(1, BATCH_SIZE // 2)
s0 = rng.choice(i0, size=min(n_half, len(i0)), replace=len(i0) < n_half)
s1 = rng.choice(i1, size=min(n_half, len(i1)), replace=len(i1) < n_half)
sel0 = np.concatenate([s0, s1])
xb0 = X_train_img[sel0].astype(np.float32) / 255.0
xb0 = (xb0 - mean_img).reshape(len(xb0), -1)
yb0 = y_train[sel0]
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", category=ConvergenceWarning, module="sklearn")
    mlp.partial_fit(xb0, yb0, classes=classes)

# =========================
# Training loop (batches)
# =========================
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", category=ConvergenceWarning, module="sklearn")
    for epoch in range(EPOCHS):
        # iterate over the whole dataset in batches (optionally shuffle)
        for xb, yb in batch_iter(X_train_img, y_train, batch_size=BATCH_SIZE, shuffle=True, rng=rng, mean_img=mean_img):
            mlp.partial_fit(xb, yb)  # no 'classes' here after the very first call

        # quick epoch eval
        preds_epoch = []
        for xb in batch_iter(X_test_img, batch_size=2048, mean_img=mean_img):
            preds_epoch.append(mlp.predict(xb))
        y_pred_epoch = np.concatenate(preds_epoch)
        print(f"Epoch {epoch+1:02d}/{EPOCHS} | test acc: {accuracy_score(y_test, y_pred_epoch):.4f}")

# =========================
# Final eval
# =========================
y_pred = []
for xb in batch_iter(X_test_img, batch_size=2048, mean_img=mean_img):
    y_pred.append(mlp.predict(xb))
y_pred = np.concatenate(y_pred)
acc = accuracy_score(y_test, y_pred)
print("Final test accuracy:", acc)

# =========================
# Inspect shapes + visualize first-layer weights
# =========================
print("mlp.coefs_[0] shape (input→hidden):", mlp.coefs_[0].shape)
print("mlp.coefs_[1] shape (hidden→output):", mlp.coefs_[1].shape)
print("mlp.intercepts_[0] shape:", mlp.intercepts_[0].shape)
print("mlp.intercepts_[1] shape:", mlp.intercepts_[1].shape)
assert mlp.coefs_[0].shape[0] == D, f"Input size mismatch {mlp.coefs_[0].shape[0]} vs {D}"

num_neurons = mlp.coefs_[0].shape[1]
rows, cols = 5, min(8, num_neurons)
fig, axes = plt.subplots(rows, cols, figsize=(10, 6))
axes = np.atleast_1d(axes).ravel()

for i, (coef, ax) in enumerate(zip(mlp.coefs_[0].T, axes)):
    img = coef_to_image(coef, H, W, C)
    ax.matshow(img, cmap=plt.cm.gray)
    ax.set_xticks(()); ax.set_yticks(())
    ax.set_title(f"N {i}", fontsize=8)

plt.tight_layout()
plt.show()
```

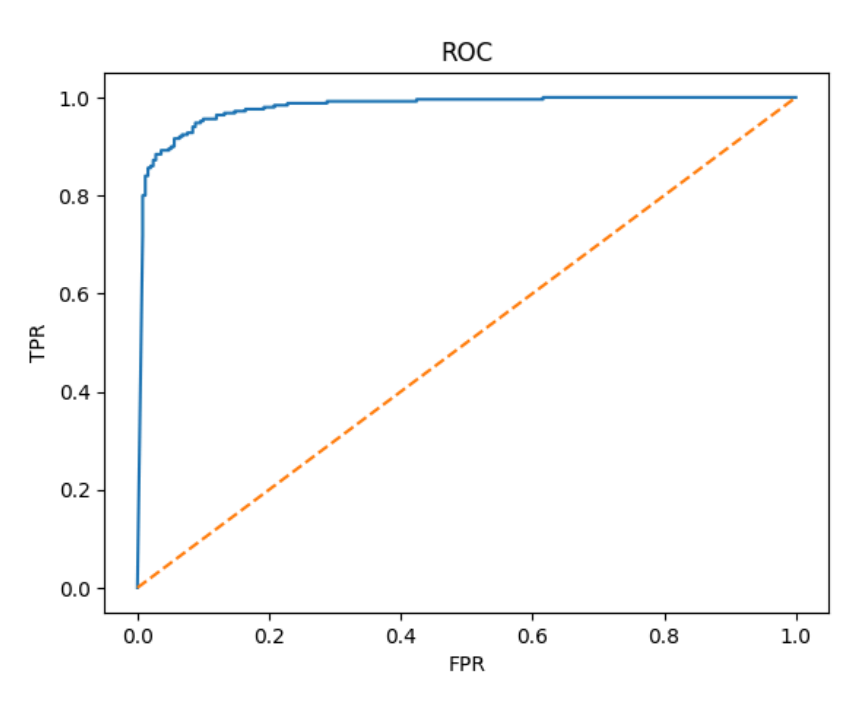
```
Train: 88345 | Test: 500 | Input dim: 4096
Epoch 01/30 | test acc: 0.9180
Epoch 02/30 | test acc: 0.9240
Epoch 03/30 | test acc: 0.9300
Epoch 04/30 | test acc: 0.9140
Epoch 05/30 | test acc: 0.9240
Epoch 06/30 | test acc: 0.9300
Epoch 07/30 | test acc: 0.9320
Epoch 08/30 | test acc: 0.9280
Epoch 09/30 | test acc: 0.9220
Epoch 10/30 | test acc: 0.9300
Epoch 11/30 | test acc: 0.9300
Epoch 12/30 | test acc: 0.9340
Epoch 13/30 | test acc: 0.9260
Epoch 14/30 | test acc: 0.9300
Epoch 15/30 | test acc: 0.9360
Epoch 16/30 | test acc: 0.9280
Epoch 17/30 | test acc: 0.9160
Epoch 18/30 | test acc: 0.9280
Epoch 19/30 | test acc: 0.9260
Epoch 20/30 | test acc: 0.9140
Epoch 21/30 | test acc: 0.9100
Epoch 22/30 | test acc: 0.9140
Epoch 23/30 | test acc: 0.9220
Epoch 24/30 | test acc: 0.9160
Epoch 25/30 | test acc: 0.9060
Epoch 26/30 | test acc: 0.9300
Epoch 27/30 | test acc: 0.9280
Epoch 28/30 | test acc: 0.9280
Epoch 29/30 | test acc: 0.9240
Epoch 30/30 | test acc: 0.9200
Final test accuracy: 0.92
mlp.coefs_[0] shape (input→hidden): (4096, 256)
mlp.coefs_[1] shape (hidden→output): (256, 128)
mlp.intercepts_[0] shape: (256,)
mlp.intercepts_[1] shape: (128,)
```



Tests

In [11]:
```python
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import numpy as np

# predictions on test (already in y_pred)
cm = confusion_matrix(y_test, y_pred)
print(cm)
print(classification_report(y_test, y_pred, digits=3))

# Probabilities + ROC
probs = []
for xb in batch_iter(X_test_img, batch_size=2048, mean_img=mean_img):
    probs.append(mlp.predict_proba(xb)[:, 1])
probs = np.concatenate(probs)

auc = roc_auc_score(y_test, probs)
print("ROC AUC:", auc)

fpr, tpr, _ = roc_curve(y_test, probs)
plt.figure()
plt.plot(fpr, tpr)
plt.plot([0,1],[0,1], linestyle="--")
plt.xlabel("FPR"); plt.ylabel("TPR"); plt.title("ROC")
plt.show()
```

```
[[219  31]
 [  9 241]]
              precision    recall  f1-score   support

           0      0.961     0.876     0.916       250
           1      0.886     0.964     0.923       250

    accuracy                          0.920       500
   macro avg      0.923     0.920     0.920       500
weighted avg      0.923     0.920     0.920       500

ROC AUC: 0.9795679999999999
```


ROC
TPR
FPR
0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.2
0.4
0.6
0.8
1.0